

ALBERI BINARI DI RICERCA

PIETRO DI LENA

DIPARTIMENTO DI INFORMATICA – SCIENZA E INGEGNERIA
UNIVERSITÀ DI BOLOGNA

ALGORITMI E STRUTTURE DI DATI
ANNO ACCADEMICO 2024/2025

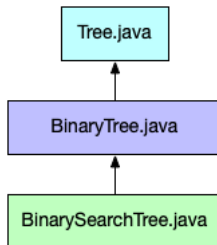


- Consideriamo la seguente interfaccia generica *Tree*

```
1 public interface Tree<K,D> {  
2     // Inserts a node storing the mapping (key,data) into the tree  
3     public void insert(K key, D data);  
4     // Returns the data stored in the first node matching key  
5     public D search(K key);  
6     // Deletes the first node matching key and returns its data  
7     public D delete(K key);  
8     // Returns the number of nodes in the tree  
9     public int size();  
10    // Returns true if the tree is empty  
11    public boolean isEmpty();  
12    // Returns a list of key values in preorder traversal  
13    public List<K> getPreorderKeys();  
14    // Returns the list of key values in postorder traversal  
15    public List<K> getPostorderKeys();  
16    // Returns the list of key values in inorder traversal  
17    public List<K> getInorderKeys();  
18    // Returns the list of key values in bfs traversal  
19    public List<K> getBFTKeys();  
20    ...  
21 }
```

- Implementare una classe *BinarySearchTree* che implementi *Tree*

- Un Albero Binario di Ricerca *estende* una struttura dati di tipo Albero Binario, che a sua volta estende una struttura dati di tipo Albero



- La classe *BinaryTree* può essere implementata come classe astratta
 - Implementa i metodi comuni a tutti gli Alberi Binari, come ad esempio i metodi di visita
 - Non implementa le operazioni specifiche per tipo di Albero Binario, come ad esempio i metodi di inserimento, ricerca e rimozione
- Obiettivo: implementare *BinarySearchTree* che estende *BinaryTree*

ULTERIORI INFORMAZIONI

- Il nodo di un Albero Binario può essere rappresentato come una classe
- Tale classe espone metodi utili per nodi di generici Alberi Binari

```
1 protected class BTreeNode {  
2     protected K key;  
3     protected D data;  
4     protected BTreeNode left, right, parent;  
5  
6     public BTreeNode predecessor() {...}  
7     public BTreeNode successor() {...}  
8     public void disconnect() {...}  
9     public void swap(BTreeNode node) {...}  
10    public boolean isRoot() {...}  
11    public boolean isLeaf() {...}  
12    public boolean isLeftChild() {...}  
13    public boolean hasTwoChildren() {...}  
14    ...  
15 }
```

La classe *BTreeNode* è una classe interna alla classe *BinaryTree*

STRUTTURA DELLA CLASSE BINARYSEARCHTREE

■ Scheletro della classe *BinarySearchTree.java*

```
1 package datastructure.tree;
2
3 public class BinarySearchTree<K extends Comparable<K>,D> extends BinaryTree<K,D> {
4     protected int size;
5
6     public BinarySearchTree() {
7         root = null;
8         size = 0;
9     }
10
11     public void    insert(K key, D data) throws IllegalStateException { ... }
12     public D      search(K key)                                     { ... }
13     public D      delete(K key)                                    { ... }
14     public int     size()                                          { ... }
15     public boolean isEmpty()                                       { ... }
16 }
```

- E' necessario implementare unicamente i metodi indicati
- Non ammettiamo l'inserimento di chiavi nulle
- Da notare il vincolo sul tipo K della chiave
 - Ogni chiave (key) deve essere *confrontabile*
 - Possiamo usare il metodo *compareTo()* per il confronto

PSEUDOCODICE DI INSERT

```
1: function INSERT(BST T, KEY k, DATA d)
2:   if k == NIL then return
3:   else INSERTNODE(NODE(k, d))
4:
5: function INSERTNODE(BST T, NODE node)
6:   prev = NIL
7:   curr = T.root
8:   while curr ≠ NIL do
9:     prev = curr
10:    if node.key < curr.key then curr = curr.left
11:    else curr = curr.right
12:  if prev == NIL then T.root = node
13:  else
14:    node.parent = prev
15:    if node.key < prev.key then prev.left = node
16:    else prev.right = node
17:    T.size = T.size + 1
```

- Se la chiave è nulla, lanciamo un'eccezione a riga 2
- La INSERTNODE ci servirà per l'estensione ad Albero AVL
- Attenzione ai confronti a riga 10 e 15

PSEUDOCODICE DI SEARCH

```
1: function SEARCH(BST  $T$ , KEY  $k$ )  $\rightarrow$  DATA
2:   if  $k == \text{NIL}$  then return NIL
3:   else
4:      $tmp = \text{SEARCHNODE}(T, k)$ 
5:     if  $tmp == \text{NIL}$  then return NIL
6:     else return  $tmp.data$ 
7:
8: function SEARCHNODE(BST  $T$ , KEY  $k$ )  $\rightarrow$  NODE
9:    $tmp = T.root$ 
10:  while  $tmp \neq \text{NIL}$  do
11:    if  $k == tmp.key$  then return  $tmp$ 
12:    else if  $k < tmp.key$  then  $tmp = tmp.left$ 
13:    else  $tmp = tmp.right$ 
14:  return NIL
```

- La SEARCHNODE ci serve per la DELETE
- Attenzione ai confronti a riga 11 e 12

PSEUDOCODICE DI DELETE

```
1: function DELETE(BST  $T$ , KEY  $k$ )  $\rightarrow$  DATA
2:   if  $k == \text{NIL}$  then return NIL
3:   else
4:      $tmp = \text{DELETENODE}(T, k)$ 
5:     if  $tmp == \text{NIL}$  then return NIL
6:     else return  $tmp.data$ 
7:
8: function DELETENODE(BST  $T$ , KEY  $k$ )  $\rightarrow$  NODE
9:    $v = \text{SEARCHNODE}(T, k)$ 
10:  if  $v \neq \text{NIL}$  then
11:    if  $v.left \neq \text{NIL}$  and  $v.right \neq \text{NIL}$  then
12:       $u = \text{PREDECESSOR}(v)$ 
13:       $\text{SWAP}(v, u)$ 
14:       $v = u$ 
15:     $\text{DISCONNECT}(T, v)$ 
16:     $T.size = T.size - 1$ 
17:  return  $v$ 
```

- La DELETENODE ci servirà per l'estensione ad Albero AVL
- Il test a riga 11 può essere effettuato invocando sul nodo v il metodo *hasTwoChildren()* della classe *BTNode*
- PREDECESSOR, SWAP e DISCONNECT sono implementate come metodi della classe *BTNode* e possono essere invocate direttamente su v

PSEUDOCODICE DI SIZE E ISEMPY

```
1: function SIZE(BST T)  
2:   return T.size  
3:  
4: function ISEMPY(BST T)  
5:   return T.size == 0
```

- Funzioni banali che agiscono sulla variabile locale *size*